

Lecture 4: Von Neumann Architecture (Stored Program Concept)

Learning Objectives

By the end of this lecture, students will be able to:

- Understand who John von Neumann was.
 - Explain the Von Neumann Architecture.
 - Understand the Stored Program Concept.
 - Identify the main components of the Von Neumann Architecture.
 - Explain how the CPU communicates with memory.
 - Understand the Fetch–Decode–Execute Cycle (Introduction).
 - Explain the Von Neumann Bottleneck.
 - Differentiate Von Neumann Architecture from Harvard Architecture (brief introduction).
-

1. Introduction

Teaching Script

Hello everyone, and welcome back to our Computer Organization and Assembly Language course.

In the previous lecture, we learned the difference between Computer Architecture and Computer Organization.

Today, we are going to study one of the most important concepts in Computer Organization:

Von Neumann Architecture.

Almost every modern computer is based on this architecture in one way or another.

If you understand this lecture well, many future topics such as CPU organization, memory, instruction execution, assembly language, and system buses will become much easier.

So let's begin by understanding what Von Neumann Architecture is and why it is so important.

2. Who Was John von Neumann?

Before discussing the architecture, briefly introduce the person behind it.

About John von Neumann

John von Neumann was one of the pioneers of modern computing.

In **1945**, he proposed a computer design in which:

- Programs are stored in memory.
- Data is also stored in memory.
- The CPU fetches instructions from memory and executes them one by one.

This became known as the **Von Neumann Architecture**.

Even today, many processors are influenced by this model.

3. What is Von Neumann Architecture?

Definition

Von Neumann Architecture is a computer architecture in which both program instructions and data are stored in the same main memory, and the CPU accesses them through a common communication path.

In simple words:

Instead of wiring every program directly into hardware, we store the program in memory just like ordinary data.

The CPU simply reads the instructions one by one and executes them.

This idea is called the **Stored Program Concept**.

4. Why Was This Architecture Revolutionary?

Before the Von Neumann model, changing a program often meant physically rewiring or reconfiguring the machine.

With the stored-program concept:

- Programs are stored in memory.
- Running a different program usually means loading different instructions into memory.
- The hardware generally remains the same.

This made computers much more flexible and practical.

5. Stored Program Concept

This is the heart of today's lecture.

Definition

The Stored Program Concept means that both instructions (programs) and data are stored together in the same memory.

For example:

Main Memory

Address	Contents
---------	----------

1000	LOAD A
1001	ADD B
1002	STORE C
1003	HALT

2000	A = 10
2001	B = 20
2002	C = 0

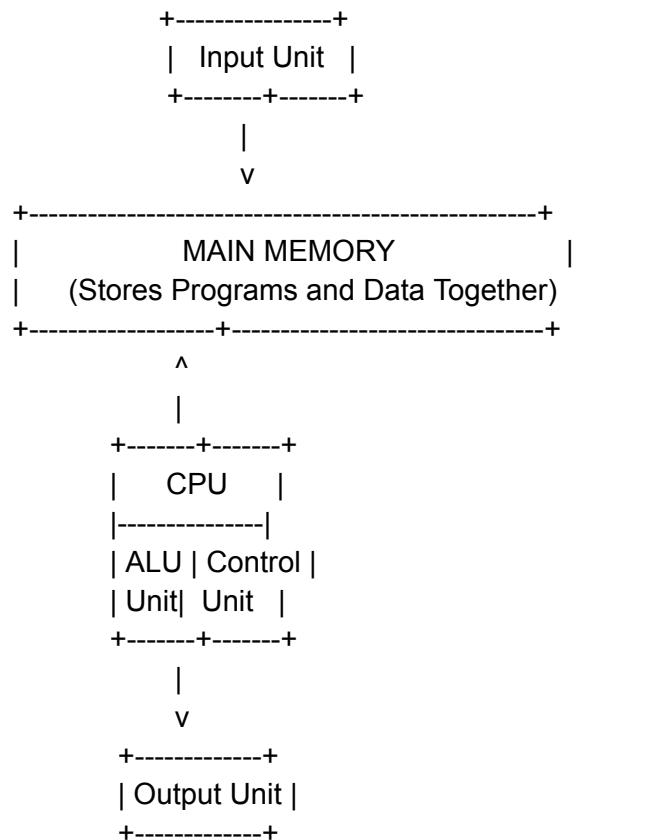
Notice that:

- Instructions are stored in memory.
- Data is also stored in memory.

The CPU reads the instructions from memory one by one.

6. Main Components of Von Neumann Architecture

The architecture consists of five major components.



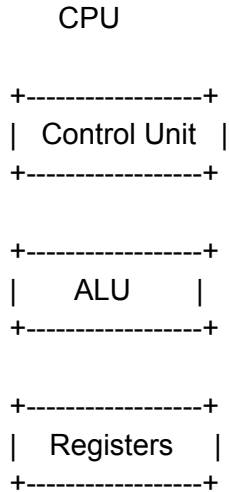
The components are:

1. Input Unit
2. Memory
3. CPU
4. Output Unit

5. System Bus (connects them)
-

7. CPU in Von Neumann Architecture

The CPU has three major parts.



Control Unit

- Fetches instructions.
- Decodes instructions.
- Controls execution.

ALU

Performs:

- Addition
- Subtraction
- Comparisons
- Logical operations

Registers

Store:

- Current instruction

- Addresses
 - Temporary values
-

8. Memory

In the Von Neumann Architecture:

Memory stores:

- Program instructions
- Variables
- Numbers
- Strings
- Intermediate results

This is different from older computer designs where programs were not stored this way.

Example:

Memory

Instruction
Instruction
Instruction
Instruction

Data
Data
Data

Both exist together in the same memory.

9. Input Unit

The Input Unit receives data from the user.

Examples:

- Keyboard
- Mouse
- Scanner
- Microphone

Example:

Keyboard



Input Unit



Memory

10. Output Unit

The Output Unit presents processed information.

Examples:

- Monitor
- Printer
- Speaker
- Projector

Example:

CPU



Output Unit



Monitor

11. System Bus

The CPU and memory must communicate.

They communicate through buses.

The three important buses are:

Data Bus

Transfers data.

Address Bus

Carries memory addresses.

Control Bus

Carries control signals such as READ and WRITE.

Together, these buses allow the CPU to communicate with memory and I/O devices.

12. Overall Working of Von Neumann Architecture

Let's understand the complete process.

Suppose we have the following simple program:

```
int a = 10;  
int b = 20;  
int c = a + b;
```

What happens?

Step 1

The program is loaded into memory.

Step 2

The CPU reads the first instruction.

Step 3

The Control Unit decodes the instruction.

Step 4

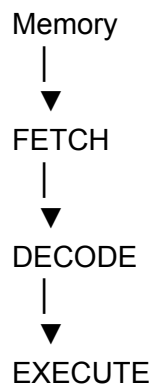
The ALU performs the required operation if needed.

Step 5

The result is stored back in memory or in a register.

13. Fetch–Decode–Execute Cycle (Introduction)

Every instruction goes through three basic stages.



Step 1 — Fetch

The CPU fetches the next instruction from memory.

Step 2 — Decode

The Control Unit determines what the instruction means.

Step 3 — Execute

The CPU performs the required operation.

Example:

Instruction:

ADD R1, R2

- Fetch the instruction.
- Decode it as an addition.
- Execute the addition.
- Store the result.

Tell students that this cycle will be studied in detail in a later lecture.

14. Example of Instruction Execution

Imagine memory contains:

Address	Instruction
---------	-------------

1000	LOAD A
1001	ADD B
1002	STORE C
1003	HALT

Execution order:

PC → 1000

LOAD A

↓

PC → 1001

ADD B

↓

PC → 1002

STORE C

↓

PC → 1003

HALT

The **Program Counter (PC)** keeps track of the address of the next instruction.

15. Advantages of Von Neumann Architecture

- Simple design.
 - Easy to understand.
 - Programs are stored in memory.
 - Easy to load and modify programs.
 - Lower hardware complexity compared to separate instruction/data memory designs.
 - Foundation of many modern computer systems.
-

16. Disadvantages — Von Neumann Bottleneck

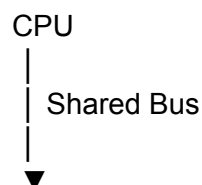
One major limitation is called the **Von Neumann Bottleneck**.

What is it?

Because **instructions and data share the same memory and communication path**, the CPU cannot fetch both simultaneously over that shared path.

This can slow down execution when the CPU spends time waiting for memory.

Illustration:



Memory

Programs
and
Data

Only one transfer can use that shared path at a given moment.

17. Real-Life Analogy

Imagine a **single-lane bridge**.

Cars carrying people

↓

Single Lane Bridge

↓

Destination

If cars are moving in both directions, only one side can use the bridge at a time.

Traffic slows down.

Similarly:

- Instructions need the bus.
- Data also needs the bus.

Since they share the same path, the CPU may have to wait.

This waiting effect is called the **Von Neumann Bottleneck**.

18. Practical Example

Suppose the CPU wants to:

- Fetch the next instruction.
- Read a value from memory.

Both requests require access to the same memory path.

Since only one transfer can occur over the shared path at a time, one request may need to wait.

24. Lecture Summary

Closing Script

Today, we studied one of the most important computer architecture models: the **Von Neumann Architecture**.

We learned that both **program instructions and data are stored together in the same memory**, which is known as the **Stored Program Concept**.

We also explored the main components of the architecture: the **Input Unit, Memory, CPU, and Output Unit**. Inside the CPU, we discussed the **Control Unit, ALU, and Registers**.

Next, we introduced the **Fetch–Decode–Execute Cycle**, which explains how the CPU processes instructions one by one.

Finally, we learned about the **Von Neumann Bottleneck**, where instructions and data compete for the same memory path, potentially slowing down execution.

In the next lecture, we'll study **Harvard Architecture**, compare it with the Von Neumann model, and understand why some modern processors use separate instruction and data memories.

Thank you for watching, and I'll see you in the next lecture.
